

DIGITALAIKART PRESENTS

The AI Coding Playbook

The 2026 Indian Developer Edition

Claude Code. Cursor. GitHub Copilot. Gemini CLI.

The tools, the prices in rupees, the workflows, the 25 prompts,
and the 30-day plan that make you the developer who ships faster.

90%

of professional developers
use AI coding agents weekly

21.9M

developers on GitHub in India,
No. 2 country worldwide

Rs. 649

Cursor's India-only plan,
billed in INR with UPI

1,000

free Gemini CLI requests
per day with a Google account

Digital Edition - Verified against official sources, September 2026

digitalkartai.shop

What's Inside

Chapter 1 | The numbers that should change how you code

90% of developers already use agents weekly - here is the verified 2026 data.

Chapter 2 | Agents changed what coding looks like

Autocomplete is dead. Spec-writing and review are the new core skills.

Chapter 3 | Claude Code - the terminal powerhouse

Install, CLAUDE.md, hooks, skills, MCP, parallel agents, pricing.

Chapter 4 | Cursor - the agent-first editor (Rs. 649 in India)

The India-only Start plan, what it includes, what it leaves out.

Chapter 5 | GitHub Copilot - the default that comes to you

Free tier, the Student plan, AI credits, and the model list.

Chapter 6 | Gemini CLI - the free workhorse

1,000 requests a day, free, open source. The budget backstop.

Chapter 7 | Beyond the big four - the 2026 tool map

Windsurf, OpenCode, Google Antigravity, JetBrains AI, OpenAI Codex - who they are for.

Chapter 8 | Choosing your stack

Five developer profiles, five stacks, with monthly costs in rupees.

Chapter 9 | The spec-first workflow

The single biggest habit upgrade: plan, execute in small diffs, verify - with a worked example.

Chapter 10 | Context files that make agents smart

How to write CLAUDE.md / GEMINI.md / rules - with a full template.

Chapter 11 | MCP - plug your agent into everything

The open standard that turns a coding agent into a team member.

Chapter 12 | Agents in your CI pipeline

Automated PR reviews and issue triage in GitHub Actions.

Chapter 13 | 25 prompts you will use every week

Copy-paste patterns for bugs, tests, reviews, docs, migrations.

Chapter 14 | Review like your job depends on it

Security, secrets, client code, and the discipline of checking.

Chapter 15 | The money chapter

Free tiers first, quota management, and the real stack maths.

Chapter 16 | When the agent goes wrong

Seven failure modes you will meet - and the fix for each.

Chapter 17 | Your first 30 days

A week-by-week adoption plan with checkpoints.

Chapter 18 | The team playbook

Rolling out AI coding tools across a team - pilots, guardrails, metrics.

Chapter 19 | What this means for your career

Augmentation beats automation - and what to actually learn.

Quick-reference checklists and cheat sheet

Before every commit, before every PR, before every subscription - plus the command cheat sheet.

Glossary

The twelve terms this book uses, in one place.

Sources

Every fact in this book, traced to its official origin.

How to read this book

If you have never used a coding agent, read chapters 1-10 in order, then start the 30-day plan in chapter 17. If you already use one tool daily, jump to the chapters for the other tools (3-6), the workflow chapters (8-11), and the prompt library (10). Every price in this book was verified against the vendor's official pricing or documentation pages in September 2026. Prices change - always re-check before you pay.

CHAPTER 1

The numbers that should change how you code

The 2026 evidence, from GitHub, JetBrains and Anthropic - not hype, published data.

Here is the uncomfortable summary: if you are a developer in India and you are not using an AI coding agent at work, you are now the minority - and the gap is compounding.

In May-July 2026, JetBrains surveyed thousands of professional developers worldwide and found that 90% of them used AI coding agents at work at least weekly, with 68% using them daily. Claude Code alone was being used at work by 39% of professional developers - nearly half of all developers in the United States. GitHub Copilot, the tool that started this era in 2023, had slipped from 29% adoption a year earlier to 21%. The market is not debating whether to adopt AI tools anymore; it is fighting over which one.

India is not a spectator in this shift. GitHub's Octoverse 2025 report found that India added more than 5.2 million developers in a single year - over 14% of all new accounts on the platform, the most of any country. India's GitHub developer base stands at 21.9 million, second only to the United States, and GitHub projects it will reach 57.5 million by 2030. India also overtook the US as the largest base of open-source contributors. And nearly 80% of new developers on GitHub use GitHub Copilot within their first week on the platform - AI assistance is now the default expectation of everyone entering the field.

What Indians actually do with AI

Anthropic published its India Country Brief in February 2026, based on roughly one million Claude.ai conversations. India accounts for 5.8% of global Claude.ai usage - second only to the United States. More striking is what that usage is: 45.2% of tasks mapped to software-related occupations, the highest share of any country analysed (Vietnam was next at 42.1%). The single most common task among Indian users was modifying existing software to correct errors, at 10.5% of all tasks. Writing new programs to meet customer requirements came second at 7.1%.

Anthropic also found that tasks estimated to take about 3.8 hours without AI were completed in roughly 15 minutes with Claude in its India sample - and that four states, Maharashtra, Tamil Nadu, Karnataka and Delhi, account for more than half of India's usage. If you code in Bengaluru, Chennai, Mumbai, Pune, Hyderabad or Delhi, you are competing directly against people who already work this way.

The vendors are building for India specifically, which tells you where they think the market is going. Cursor launched an India-only plan in July 2026 at Rs. 649 per month, payable by UPI, after its Indian user base tripled in a year to more than 3 million developers - its third-largest market, and the one with the most agent requests per developer anywhere in the world. Anthropic opened a Bengaluru office in February 2026 and reported that India is its second-largest Claude.ai market.

The two-developer gap

Two engineers on the same team, same salary, same stack. One treats AI tools as autocomplete and uses maybe 20% of their capability. The other writes tight specs, reviews every diff, and ships two features a week. In performance reviews, the difference looks like talent. It is not. It is workflow - and workflow is learnable in a month. That is what the rest of this book teaches.

One more number for scale. In its February 2026 funding announcement, Anthropic said Claude Code's run-rate revenue had crossed \$2.5 billion, that its weekly active users had doubled since January 1, and that an outside analysis estimated about 4% of all public GitHub commits worldwide were being authored by Claude Code. Whatever you think of the technology, the industry has voted with its keyboard.

CHAPTER 2

Agents changed what coding looks like

Why the skill that mattered in 2023 (prompting autocomplete) is not the skill that matters now.

The first generation of AI coding tools - GitHub Copilot as launched in 2023 - was autocomplete. It predicted your next few lines as you typed. The value flowed to people who wrote clean, predictable code, because the model could continue it.

Today's tools are agents. You describe a task; the agent reads your repository, plans an approach, edits multiple files, runs your tests and commands, checks its own output, and iterates until it finishes or asks you a question. Claude Code runs in your terminal and can spawn parallel sub-agents for different parts of a task. Cursor's agent mode edits across files with full project context. Copilot's agent takes a GitHub issue and opens a pull request. The unit of work is no longer the line - it is the task.

This flips where your skill and time should go. With autocomplete, you typed more. With agents, the scarce inputs are:

- **A clear specification.** The agent is only as good as the task description. Vague prompt, vague code.
- **Curated context.** Which files, which conventions, which constraints - fed to the agent before it starts.
- **Review discipline.** The agent writes code fast; you are now the quality gate, so reading diffs becomes the core skill.
- **Verification loops.** Tests the agent must pass before it stops, so it self-corrects instead of handing you hopeful code.

Anthropic's own usage data shows this shift happening. Between October 2025 and April 2026, the share of Claude Code sessions focused primarily on fixing broken code fell from 33% to 19%, while sessions for operating software and writing rose - people stopped using agents mainly to repair bad AI output and started using them to build. That happens when the workflow is right.

What did not change

The rest of the job still belongs to you. Deciding what to build, negotiating requirements, understanding why the 2019 system behaves strangely in production, talking to the teammate whose module you are about to touch, choosing between fast-now and clean-later - none of that is agent work, and all of it still decides who is valuable on a team. What changed is the floor: the mechanical half of engineering - typing out boilerplate, chasing syntax errors, writing the docs nobody wanted to write - is now nearly free. The strategic half got more valuable, not less, because more teams can now ship more software and the differentiation has moved up the stack.

The honest caveats

Agents are not magic, and this book will not pretend otherwise. They hallucinate APIs that do not exist. They write plausible tests that test nothing. They will confidently refactor the wrong file if your spec is ambiguous. The developers who win with them are the ones who treat the agent like a brilliant, overconfident junior engineer: given clear instructions and tight review, extraordinary; given vague instructions and blind trust, a liability. Chapters 9, 10 and 14 are about becoming that manager.

The other caveat is cost. These tools bill by usage beyond your plan's allowance, and heavy agent users can burn through quotas in days. Chapter 12 covers the free tiers, the rupee-priced options and the quota maths, so you do not get a surprise card statement.

CHAPTER 3

Claude Code - the terminal powerhouse

The most widely adopted AI coding tool of 2026 - and the one that rewards engineers who live in the terminal.

Claude Code is Anthropic's agentic coding tool. It reads your codebase, edits files, runs commands, creates git commits and pull requests, and integrates with your development tools. It runs in your terminal, as extensions inside VS Code and JetBrains IDEs, as a standalone desktop app, and in the browser at claude.ai/code - all surfaces share the same engine, your settings and project memory.

Getting started

On macOS, Linux or WSL, the official install is a one-liner:

```
curl -fsSL https://claude.ai/install.sh | bash
```

Then cd into any project and type **claude**. You will be prompted to log in with a Claude subscription or an Anthropic Console account. Native installs update themselves in the background. Windows installs via WinGet or PowerShell; Git for Windows is recommended so Claude Code can use the Bash tool.

What it costs

Plan	Price (USD)	What you get for coding
Claude Free	\$0	Chat, artifacts, web search - but no Claude Code
Claude Pro	\$20/mo (\$17/mo billed annually at \$200/yr)	Claude Code, Cowork, Design, Science, projects, more models
Claude Max 5x / 20x	From \$100/mo	5x or 20x Pro usage, higher output limits, priority access

Usage is metered in rolling five-hour session windows plus weekly limits - you discover your ceiling by hitting it mid-task, which is the most common complaint about Pro. Heavy agent users graduate to Max. Prices exclude tax.

The five features that separate users from power users

- 1 CLAUDE.md - project memory.** A markdown file at your project root that Claude Code reads at the start of every session. Coding standards, architecture decisions, preferred libraries, review checklists - written once, applied forever. Claude also builds automatic memory as it works. Chapter 9 shows you how to write a good one.
- 2 Skills.** Package repeatable workflows as shareable commands - a `/review-pr` or `/deploy-staging` your whole team can run.
- 3 Hooks.** Shell commands that fire before or after Claude Code actions - auto-format after every file edit, run lint before every commit. Quality control that runs even when you forget.
- 4 MCP - the Model Context Protocol.** An open standard for connecting the agent to external systems. Read design docs in Google Drive, update Jira tickets, pull context from Slack - or your own custom tooling.

- 5 **Parallel agents.** Spawn multiple agents working on different parts of a task at once, coordinated by a lead agent; background agents run long tasks while you keep working. Routines run on a schedule - morning PR reviews, overnight CI-failure analysis - in the cloud, even when your computer is off.

Claude Code also composes with the Unix philosophy: pipe logs into it, run it in CI through GitHub Actions or GitLab CI/CD, script it headless. In Anthropic's February 2026 funding announcement, Claude Code's run-rate revenue had crossed \$2.5 billion and weekly active users had doubled since New Year's Day. In JetBrains' mid-2026 survey it was the most widely adopted AI coding tool at work - 39% of professional developers, and the primary tool for 31% of them.

The five tasks it is built for

The documentation pitches Claude Code at exactly the tedious work that eats a developer's day: writing tests for untested code, fixing lint errors across a project, resolving merge conflicts, updating dependencies, writing release notes. Its own example command is the whole philosophy in one line: 'write tests for the auth module, run them, and fix any failures'. The agent plans, edits across files, runs the commands and iterates until the loop is green - you review the result. For bugs, paste the error message; it traces the issue through the codebase, identifies the root cause and implements the fix. For git, it stages changes, writes commit messages, creates branches and opens pull requests straight from the terminal.

Who should pick it

You do real multi-file work: refactors, migrations, debugging across a codebase, test suites. You are comfortable in a terminal. You want the agent to run commands and verify its own work. If your coding time is mostly reading unfamiliar client codebases - common in Indian IT services - Claude Code's codebase-wide comprehension is the strongest of any tool in this book.

CHAPTER 4

Cursor - the agent-first editor (Rs. 649 in India)

A VS Code fork rebuilt around AI - and the only major vendor with a rupee-priced plan for India.

Cursor is an AI-native IDE - a fork of VS Code where the agent is built into the editor itself. It indexes your project, edits across multiple files from a single instruction, and offers Agent, Plan and Ask modes. If you already live in VS Code, the migration is nearly effortless: same keybindings, same extensions, agent everywhere.

The plan that matters for India: Start

On 28 July 2026, Cursor launched **Start**, an India-only subscription, after its Indian user base tripled in a year to more than 3 million developers. The details, from Cursor's own documentation:

- **Rs. 649 per month, tax inclusive** - the number you see is the number that leaves your account.
- **Paid in INR with UPI**, or Indian credit/debit cards (3D Secure). UPI is recommended for reliable renewals.
- Includes **Grok 4.6, Grok 4.5 and Composer 2.5** (the Cursor models pool) with a larger monthly usage pool than the free Hobby plan, plus **Cloud Agents** (long-running agents that build, test and open PRs) and **Cursor for iOS** with remote control.
- Models run at fixed medium effort in non-fast mode. **Not included:** third-party models (OpenAI, Anthropic, Google), Bugbot, on-demand overage spend, the SDK, Automations. Hit the cap and AI features pause until the next cycle - no surprise bills.

- You verify an Indian phone number to subscribe. Once you upgrade to a global Pro plan, you cannot downgrade back to Start.

The rest of the ladder

Plan	Price	Third-party model usage included
Hobby	Free	Limited agent requests; Composer access
Start (India only)	Rs. 649/mo, tax incl.	Not included - Cursor models only
Pro	\$20/mo	\$20 of other-model usage
Pro+	\$60/mo	\$70 of other-model usage
Ultra	\$200/mo	\$400 of other-model usage
Teams	\$40/user/mo	Team allowance (Premium at \$120)

All prices except Start are shown before tax and bill in USD - your bank's forex markup of roughly 2-3.5% applies. Pro and above add Bugbot (agentic code review), Auto mode and on-demand usage.

Privacy mode

Cursor's pricing page commits that with privacy mode enabled, your code is not used for training by Cursor or its model providers. For freelancers handling client code or teams under NDA, turn it on before your first agent request - it is in settings, and admins can enforce it team-wide. The same discipline applies to every tool in this book: know your vendor's training policy before you paste client code. And note Cursor's own warning - subscriptions are sold only through cursor.com; there are no authorised resellers.

Agent, Plan and Ask - the three postures

Cursor's loop has three modes, and choosing between them is most of the skill. **Ask** answers questions about your code without editing anything - your exploration mode. **Plan** researches the codebase and presents an implementation plan for your approval without writing code - the spec-first workflow's Move 2, built into the editor. **Agent** executes: edits files, runs commands, iterates. The common failure is living in Agent when half your sessions should be Plan - correcting a plan costs a sentence, unwinding a wrong implementation costs an afternoon. Tab completions stay instant on paid plans, so the autocomplete muscle memory you already have keeps working underneath all three.

Who should pick it

You want the fastest path from VS Code to agentic coding, and you want to pay in rupees with UPI. Start at Rs. 649 is the best value agent-first IDE deal available in India right now - with the honest caveat that it runs Cursor's own models, not Claude or GPT. When you need frontier third-party models inside the editor, that is what Pro is for.

CHAPTER 5

GitHub Copilot - the default that comes to you

The tool most Indian developers will touch first - because it is already inside VS Code and GitHub.

GitHub Copilot is the incumbent. It ships as extensions in VS Code and JetBrains IDEs, it is woven into github.com - open an issue, click start work, and Copilot plans the implementation and opens a PR - and every plan now includes Copilot CLI and the Copilot app. GitHub's Octoverse 2025 report found nearly 80% of new developers on the platform use Copilot within their first week.

The plans, verified from GitHub's documentation

Plan	Price (USD)	AI credits / limits
Copilot Free	\$0	Allowance of credits; 2,000 completions/month; auto model selection only
Copilot Student	Free	For verified students - Copilot's features at no cost
Copilot Pro	\$10/mo	1,500 total monthly credits (1,000 base + 500 flex); model selection
Copilot Pro+	\$39/mo	7,000 total monthly credits; premium models
Copilot Max	\$100/mo	20,000 total monthly credits; priority access
Copilot Business	\$19/seat/mo	1,900 credits per user; org policy control
Copilot Enterprise	\$39/seat/mo	3,900 credits per user; enterprise features

Two things to note. First, code completions and next-edit suggestions are unlimited on paid plans - the credits meter agent-style work (chats, agent mode, coding agent). Second, billing beyond your included credits is usage-based: GitHub documents additional usage at \$0.01 per credit in organisations.

The quiet superpower: the model list

Copilot Pro and above let you route between frontier models from every major lab - the documented list spans GPT-6 Astra and the GPT-5.x family, Claude (Opus, Sonnet, Haiku), Gemini 3.x Flash, Grok and more. You are not marrying one model; you are choosing per task. This is why Copilot remains the safe default in large Indian IT services and BFSI shops - per-seat pricing, existing Microsoft enterprise agreements, and admin policy controls their compliance teams already trust.

What every plan includes

Whatever you pay, every Copilot plan now includes Copilot CLI and the Copilot app - the assistant follows you off the editor page. Inline suggestions and next-edit suggestions are real-time and, on paid plans, unlimited; the AI credits meter the chat and agent workloads. Copilot Chat runs in IDEs, inline, through slash commands, in GitHub Mobile, in Windows Terminal and on github.com, and paid tiers add chat skills. Agent mode and cloud agents escalate from completion to multi-file execution, and Copilot code review can be pointed at a selection right inside VS Code even on the free tier.

Who should pick it

You want the lowest-friction start (it is already in your editor), you are a verified student (free), or your organisation already runs it - in which case your real job is to use the credits you are already paying for. Pro at \$10 with 1,500 credits is the cheapest credible paid tier among the majors.

CHAPTER 6

Gemini CLI - the free workhorse

An open-source agent from Google with a free tier so generous it should be in every Indian developer's toolbox.

Gemini CLI is Google's open-source (Apache 2.0) AI agent for the terminal. Its headline number is absurd for a free product: with a personal Google account, the free tier gives you **60 requests per minute and 1,000 requests per day**, running on Gemini 2.5 Pro with a one-million-token context window. For students, freelancers on thin months, and side projects, this is the difference between using agents daily and not.

Install is a single command - Node.js 20 or higher required:

```
npm install -g @google/gemini-cli
gemini # then choose 'Login with Google'
```

There is also an API-key route (100 free requests/day, model selection, usage-based upgrades) and a Vertex AI route for enterprises.

What it can do

- **GEMINI.md context files** - the same idea as CLAUDE.md: persistent project context read every session (chapter 10).
- **Built-in tools** - Google Search grounding for current information, file operations, shell commands, web fetching.
- **MCP support** - connect Jira, Slack, databases or custom tool servers, same open standard as Claude Code.
- **GitHub integration** - an official GitHub Action for automated PR reviews, issue triage, and on-demand help via @gemini-cli.
- **Scriptable** - non-interactive mode with JSON output for pipelines; checkpointing to save and resume long sessions.

Where does a free tool fit when you pay for others? Everywhere you would rather not spend quota: exploratory analysis, one-off scripts, learning agentic workflows, docs generation, and as the backstop when your paid tool's five-hour window is exhausted at 11pm before a demo. Running two CLIs side by side costs you nothing but a terminal tab.

The commands worth knowing

A few flags do most of the work once you are past the interactive start: **gemini -p 'Explain the architecture of this codebase'** asks a one-off question non-interactively; adding **--output-format json** gives structured output for scripts. **-m gemini-2.5-flash** switches model when you want speed over depth. **--include-directories ../lib,../docs** pulls sibling folders into context when the code you are changing lives in another repo. Checkpointing saves and resumes long sessions, and the **/bug** command files issues straight from the CLI.

Who should pick it

Students and freshers (Rs. 0, generous limits, full agentic workflow to practise), freelancers between projects, and any developer who wants a pressure-release valve on their paid quotas. Pair it with one paid tool rather than instead of one - the frontier models behind the paid tools still lead on hard tasks.

CHAPTER 7

Beyond the big four - the 2026 tool map

Five more names you will hear this year, and the niche each one actually owns.

The four tools in chapters 3-6 cover most Indian developers' needs. But the market is crowded, colleagues will swear by others, and two of these names matter unusually in India. Here is the map, with adoption data from JetBrains' mid-2026 survey of professional developers.

Tool	Who makes it	Work adoption (mid-2026)	The niche it owns
Google Antigravity	Google	6% globally; 15% in India	Agentic IDE; strongest traction in India, tied with Cursor for third place
OpenCode	Open source	7% globally	Open-source terminal agent; 42% awareness without a big company behind it
JetBrains AI / Junie	JetBrains	~9% globally	For teams that live in IntelliJ-family IDEs and want AI without leaving them
OpenAI Codex	OpenAI	3% in Jan 2026	Cloud coding agent from the ChatGPT maker; awareness growing fast
Windsurf	Windsurf	-	AI IDE alternative popular with teams that need strict deployment control

Two observations from that table. First, India is unusual: Google Antigravity's worldwide adoption is 6%, but in India it runs at 15% - tied with Cursor for third place, up from 10% in January. If a teammate uses it, that is why. Second, openness now competes: OpenCode reached 7% adoption with 42% mind share and no marketing budget behind it, the way Gemini CLI reaches you free - open-source agents are a real track, not a hobbyist one.

How to treat the long tail: you do not need a fifth tool. Pick a primary (one of the big four), keep one free secondary (Gemini CLI), and try others only when a specific need appears - your IDE already being IntelliJ (JetBrains AI), an organisational mandate (Antigravity via your Google-centric stack), or procurement constraints that favour open source (OpenCode). Every tool you add costs context switching, and the workflow in chapter 9 transfers between them anyway - that is the point of learning the workflow rather than the tool.

CHAPTER 8

Choosing your stack

Five developer profiles, five honest recommendations - with the monthly bill in rupees.

There is no single best stack; there is a best stack for your situation. Below, USD plans are converted at roughly Rs. 83/USD plus a 3% forex charge - your card's rate will vary slightly.

1. The student or fresher - Rs. 0/month

Start with **Gemini CLI** (1,000 free requests a day) plus **GitHub Copilot Free** (2,000 completions a month). Verify your student status and Copilot Student unlocks the full feature set free. Add Cursor's Hobby plan for agent practice. Your constraint is money, and the free stack is better today than the paid stack of two years ago. What you must not skip: the review discipline in chapter 14 - bad habits learned on free tools follow you into your first job.

2. The first-job engineer (0-3 years, IT services or product) - about Rs. 830/month

Ask what your employer provides before you spend anything - many Indian IT firms and GCCs already hold Copilot Business seats or are piloting Claude Code; JetBrains found Copilot adoption of 40% in companies over 5,000 employees. If nothing is provided: **Copilot Pro** (\$10, about Rs. 830) for daily autocomplete, chat and model choice, plus Gemini CLI free for overflow. Your career case: get comfortable directing agents early - the fresher cohort that reviews AI output well becomes the senior cohort that ships fastest.

3. The startup / product engineer - about Rs. 1,500-2,500/month

Cursor Start at Rs. 649 (rupee billing, UPI, generous Cursor-model usage) or **Claude Pro at \$20** (about Rs. 1,660, includes Claude Code), depending on whether you prefer an editor or a terminal. Ambitious teams run both: Cursor for in-editor flow, Claude Code for repo-wide work. The stack maths: if this saves you one productive hour a day and your time is worth even Rs. 500 an hour to your company, a Rs. 4,000/month stack pays for itself in the first week.

4. The freelancer - Rs. 650-2,000/month, client-dependent

Your added constraint is **client confidentiality**. Check every NDA before any client code touches any tool; prefer vendors with training opt-outs (Cursor privacy mode, enterprise configurations) and keep sensitive repos on the tool you have verified. Cursor Start's tax-inclusive rupee pricing also makes it the cleanest line item to pass through as a business expense.

5. The team lead / architect - per-seat, from the team's budget

Your question is not which tool but how to roll one out: pick one primary vendor for predictable billing (Copilot Business at \$19/seat, Cursor Teams at \$40/user, or Claude through enterprise channels), enable privacy and audit controls from day one, write the team CLAUDE.md together (chapter 10), and pilot on one real project for 4-6 weeks measuring review-rework rate before standardising. JetBrains' data shows team adoption is now the norm, not the experiment.

Profile	Stack	Monthly cost
Student / fresher	Gemini CLI + Copilot Free/Student + Cursor Hobby	Rs. 0
First-job engineer	Copilot Pro + Gemini CLI	~Rs. 830
Startup engineer	Cursor Start, or Claude Pro; or both	Rs. 649-2,500
Freelancer	Cursor Start (privacy mode) or Claude Pro	Rs. 650-1,700
Team lead	Copilot Business / Cursor Teams / Claude enterprise	\$19-40 per seat

CHAPTER 9

The spec-first workflow

The single biggest habit upgrade in this book - plan, execute in small diffs, verify, commit.

Most disappointing AI coding sessions are not the model's fault. They are a vague prompt meeting a large codebase with no verification loop. The fix is a workflow, and it has five moves.

Move 1: Explore before you ask

Before asking an agent to change anything, ask it to explain. Point it at the module, request a map of how the pieces connect, where the pattern you care about is used, and what the conventions are. Claude Code and Gemini CLI excel at this with large or unfamiliar codebases - the single most common task in Anthropic's India data was modifying existing software, which is exactly the case where understanding must come first. Now your edits are grounded in reality instead of the agent's guess of it.

Move 2: Write the spec, then let the agent plan

A good spec answers four things: the goal in one sentence, the files or modules in scope, the constraints (style guide, error handling, backwards compatibility), and how you will verify success (which tests must pass, what command proves it works). Then ask the agent to present a plan - Cursor's Plan mode and Claude Code's plan review exist precisely for this - and interrogate the plan before letting it write a single line. Correcting a plan costs a sentence; correcting a wrong implementation costs an afternoon.

Move 3: Small diffs, always

Ask for one coherent change at a time. 'Refactor authentication' produces 40 files of unreviewable diff. 'Extract token refresh into a helper with tests, no behaviour change' produces 30 minutes of reviewable work. Small diffs keep you in command of the codebase - the moment you stop understanding what changed, you have traded your engineering judgement for velocity you cannot audit.

Move 4: Make the agent verify itself

End every task instruction with the loop: run the tests, run the linter, fix what fails, then show me the diff. Agents that check their own output self-correct; agents that only generate hand you hopeful code. Hook your linter and formatter to run automatically after every edit (Claude Code hooks, chapter 3) so the floor is enforced even when you forget.

Move 5: Review, then commit with intent

Read every diff before accepting it - the review checklist in chapter 14 takes about three minutes per diff and catches nearly everything. Then let the agent write the commit message and PR description from the actual diff: accurate, boring, and free. Claude Code stages changes and creates commits and PRs directly from the terminal.

TASK: Add rate limiting to the login endpoint

GOAL: Stop credential-stuffing attacks; lock each account for 15 minutes after 5 failed logins within 10 minutes.

SCOPE: src/api/auth/login.ts and a new helper in src/services/ratelimit.ts. Do not touch other endpoints. Do not add a dependency - use the existing Redis client (src/db/redis.ts).

CONSTRAINTS: TypeScript strict; follow the error-handling convention in CLAUDE.md (AppError subclasses). The lockout must work across all app instances (hence Redis, not in-memory).

VERIFY: npm test must pass, and add tests for: 5th failure locks, 4th does not, lock expires after 15 minutes (mock the clock), success resets the counter. Run the suite and show me the results before we finish.

Read that spec again and notice what it does: the agent cannot drift into the wrong files (scope), cannot pick a wrong library (constraint), cannot claim success without proof (verify), and knows the house style (context file). A minute of typing buys you a reviewable, verifiable change - versus the 'add rate limiting to login' one-liner, which is how 40-file diffs happen.

A week with the loop (illustrative)

To see the whole thing in motion, here is a representative week for a mid-level engineer on a product team who has internalised this book - Monday to Friday, tickets and all.

- **Monday.** Sprint planning hands over three tickets. Before coding: prompt 3 on the module each ticket touches, refreshing the context file with anything new the agent's archaeology uncovers.
- **Tuesday.** Ticket one is a bug: prompt 7 - failing test first, then the fix, then the diff. Ten minutes of review, merged.
- **Wednesday.** Ticket two is a feature: prompt 17 for the plan, interrogate it, then prompt 18 twice for two thin slices. The CI agent (chapter 12) posts its findings before the human review; one finding is real - an unvalidated input the spec had not thought of.
- **Thursday.** Ticket three is a migration nobody wants: prompt 20. The plan lands in the team channel for discussion rather than the codebase - the risky step is named and deferred to a quieter release window.
- **Friday.** The agent writes the PR descriptions (prompt 22), the week's docs update, and a summary of review findings that becomes next week's context-file additions. Total review time for the week: maybe three hours. Total code written by hand: almost none. Total understanding required: all of it.

That is the honest shape of agentic coding done well: your keyboard time falls, your reading and deciding time rises, and the ratio of shipped-to-effort changes completely. Notice also what is absent - no late-night heroics, no mystery code in Monday's standup.

The workflow in one line

Explore - specify - plan - small diff - self-verify - review - commit. If you adopt nothing else from this book, adopt this loop and run it on one real ticket tomorrow.

A note on sessions: paid plans meter usage in windows (Claude's five-hour rolling window plus weekly caps; Cursor's monthly pool that resets each cycle). Batch related work into one session so context - and your quota - is not spent re-reading the same files.

CHAPTER 10

Context files that make agents smart

CLAUDE.md, GEMINI.md, rules files - write once, and every future session starts smarter.

A context file is a markdown file in your project root that the agent reads at the start of every session. Claude Code reads CLAUDE.md; Gemini CLI reads GEMINI.md; Cursor uses project rules. Same idea everywhere: you are writing the onboarding doc you would give a brilliant new teammate who has amnesia between sessions.

A bad context file is a wall of prose nobody (including the agent) reads. A good one is short, structured, and full of decisions rather than descriptions. Here is a template that works for almost any repo - adapt the bracketed parts, delete what does not apply.

```
# CLAUDE.md

## Project
[1-2 sentences: what this is, who uses it.]

## Commands
- Run tests: `npm test`
- Run single test: `npm test -- path/to/file`
- Lint + format: `npm run lint && npm run format`
- Local server: `npm run dev`

## Architecture
- `src/api` - REST routes; handlers stay thin, logic in services
- `src/services` - business logic; no framework imports allowed
- `src/db` - queries only; migrations live in `migrations/`
- Data flow: route -> service -> db. Never route -> db.

## Conventions
- TypeScript strict; no `any` without a comment saying why
- Errors: throw `AppError` subclasses; never raw strings
- Tests colocated as `*.test.ts`; every bug fix ships with a test
- Commit style: conventional commits (feat:, fix:, chore:)

## Do not
- Do not edit `generated/` files
- Do not add dependencies without asking
- Do not 'fix' the payment retry logic - ask first; it is load-bearing

## Reviewer priorities
1. Security (auth, input validation) 2. Test coverage 3. Readability
```

Why each section earns its place

- **Commands** lets the agent run your test/lint loop itself - the foundation of self-verification.
- **Architecture** prevents the classic failure: agent puts SQL in a route because it never learned your layers.
- **Conventions** is where you encode taste: error handling, testing, commit style - things reviews keep catching.
- **Do not** is the highest-value section in the file. Guard rails around generated code, load-bearing logic and dangerous paths.
- **Reviewer priorities** tells the agent what you care about when it self-reviews - your order, not the internet's.

Keep it under a page. Update it every time a review catches something the agent could have known - the file should evolve into your team's collective memory. Claude Code additionally builds automatic memory as it works; Gemini CLI checkpoints long sessions. Neither replaces a maintained context file, because neither knows what your team decided on purpose.

One rule for teams: commit the context file to the repo. Every teammate's session - and every CI run - now starts with the same understanding. That is how one person's conventions quietly become everyone's default.

CHAPTER 11

MCP - plug your agent into everything

The Model Context Protocol is why agents stopped being autocomplete and started being teammates.

MCP - the Model Context Protocol - is an open standard for connecting AI tools to external data sources and systems. Both Claude Code and Gemini CLI speak it, and tool vendors across the industry have adopted it. In practice, an MCP server is a connector: you configure it once, and your agent can then read, query or update that system on request, the way a teammate with login credentials would.

What that looks like in real use, from the vendors' own documentation: with MCP, Claude Code can read your design docs in Google Drive, update tickets in Jira, and pull context from Slack - or use your company's own internal tooling. Gemini CLI's documentation shows the same pattern in its command syntax: ask GitHub to list your open pull requests, ask Slack to send a summary of today's commits to the dev channel, ask a database server to run a query. The agent stops being a code generator that knows only your filesystem, and starts being an operator that knows your project.

Why this matters more than model quality

- **Context stops being copy-paste.** Instead of pasting the ticket text into the prompt, the agent reads the ticket itself - no staleness, no truncation.
- **Verification gets richer.** The agent can check the CI status, look at the incident dashboard, or read the runbook before it proposes a fix.
- **Your internal tools become AI-capable without writing AI code.** One well-scoped MCP server (a database reader, a deployment trigger) upgrades every agent on the team at once.
- **It is vendor-neutral.** The same servers work across tools that support the standard - you are not locked into wiring your stack to one vendor.

Starting small, safely

Start with one or two servers, not ten. The highest-value first server for most teams is the GitHub one: the agent sees your PRs, your review comments and your CI results, which makes prompt 14 (the pre-PR review) dramatically better informed. Add Jira or your tracker next, then Slack last - write access to team chat is the one to think twice about. And apply the security thinking from chapter 14 (the review chapter) to MCP too: a connector that can write to a system is a privilege, so grant it read-only first and widen only when a real need appears.

CHAPTER 12

Agents in your CI pipeline

PR reviews and issue triage that run while you sleep - the official patterns.

The same agents you use interactively have documented, non-interactive modes for automation - this is the quiet half of the agentic shift. Claude Code's documentation covers running it in CI, including GitHub Actions and GitLab CI/CD, for automated code review and issue triage. Gemini CLI ships an official GitHub Action for the same two jobs - pull-request review with contextual feedback, and issue labelling and prioritisation - plus on-demand help by mentioning the agent in an issue or PR. GitHub's own Copilot completes the set: open an issue, start work, and its coding

agent plans the implementation and opens a pull request.

A sane adoption ladder for CI agents

- 1 **PR summaries.** Zero risk, immediate value: every PR gets an accurate description generated from the diff (prompt 22's pattern, run by a machine).
- 2 **Review assistance, human decides.** The CI agent posts findings as comments; a human approves. You get a tireless second reviewer on every PR - use prompt 14's severity ranking.
- 3 **Issue triage.** Labels, prioritisation, duplicate detection - the queue maintenance nobody wants to do.
- 4 **Automated fixes for the boring classes.** Dependency bumps, lint migrations, docs sync - high-volume, low-judgement work, always merged through normal review.

Keep humans as the final approvers on anything that touches production, security or architecture. The failure mode of CI agents is not that they are wrong - it is that they are right often enough that the team stops reading. Pair automation with the review checklist from chapter 14, and measure it: track how many agent findings are acted on versus dismissed. If the dismiss rate climbs, tighten the agent's instructions - noisy automation gets muted, and a muted reviewer is worse than none.

CHAPTER 13

25 prompts you will use every week

Copy-paste patterns for the work you actually do - bugs, tests, reviews, docs, migrations.

These are patterns, not scripts: replace the bracketed parts, and add your context file (chapter 10) before running any of them. They work in Claude Code, Cursor's chat and agent modes, Copilot Chat, and Gemini CLI - phrased slightly terminal-flavoured, which is the lowest common denominator.

Adapting these to your stack

- **Add your context file first.** Every prompt below assumes the agent already knows your conventions; it reads them from CLAUDE.md / rules (chapter 10).
- **Give evidence, not descriptions.** Where a prompt says [paste error] or [file path], paste the real thing - the agent works better on the artifact than on your summary of it.
- **Name the guardrails you care about.** 'Do not refactor anything unrelated' and 'do not add dependencies' are the two that save the most review time; add your own to the list.
- **Ask for plans before code on anything non-trivial.** Append 'present your plan first, do not write code until I approve' to prompts 17-20 when stakes are high.

Understanding a codebase (prompts 1-5)

PROMPT 1 | The codebase map

```
Explore this repository and give me: (1) a one-paragraph summary of what it does, (2) the directory structure with what each top-level folder is responsible for, (3) the 5 most important files and why. Do not suggest changes yet - I want to understand it first.
```

Run this the day you join a project. It converts 'lost for a week' into 'oriented in an hour'.

PROMPT 2 | The feature trace

Trace how a [user signup / payment / similar request] flows through this codebase, entry point to database. List every file and function in order, one line each on what it does.

Gives you the mental map you normally build by grep-and-pray over weeks.

PROMPT 3 | The convention detector

Look at 5 recent commits and the code they touched. Infer this project's unwritten conventions - naming, error handling, test structure, comment style - and list them as rules I can add to my context file.

Builds chapter 10's context file for a repo you inherited, from evidence rather than assumption.

PROMPT 4 | The dependency explainer

Explain what [dependency/package name] is doing in this project: where it is used, what it provides, what breaks if we remove it. Check package files and imports - do not guess from the name.

The anti-hallucination phrasing ('do not guess') matters - agents will otherwise fill gaps confidently.

PROMPT 5 | The legacy code archaeologist

This function has no comments and no one on the team knows why it exists: [paste or file path]. Explain what it does step by step, what its inputs and outputs are, and what a safe refactor of it would look like. Flag anything that looks like a deliberate workaround.

Anthropic's India data shows 'modifying existing software' is already your most common AI task. This is that, done properly.

Fixing bugs (prompts 6-9)

PROMPT 6 | The error triage

I am getting this error: [paste full stack trace]. Here is what I was doing: [context]. Find the root cause in this codebase - do not just explain the error message generally, show me the exact line(s) where it originates and why.

Paste the whole trace. The first line of the traceback is rarely the interesting one.

PROMPT 7 | The repro-first bug hunt

Bug: [describe behaviour vs expected]. First, write a failing test that reproduces this bug. Show me the test and confirm it fails. Then find the root cause, fix it minimally, and make the test pass. Do not refactor anything unrelated.

The failing test proves the bug exists and later proves you fixed it. 'Fix minimally' stops drive-by refactors.

PROMPT 8 | The intermittent bug diff

This bug happens only [sometimes / under load / on first request]: [description]. List the 5 most likely causes in this specific codebase - race conditions, caching, state, environment - with the file and line for each, ranked by likelihood. I will decide where to look.

You keep the judgement; the agent does the systematic enumeration you are too busy for.

PROMPT 9 | The production log reader

Here are the logs around the incident: [paste]. Build a timeline of what happened, correlate entries across services, and tell me the first anomaly in the sequence.

Feed it as context, never let it touch production itself. Reading is safe; writing is not.

Writing tests (prompts 10-12)

PROMPT 10 | The coverage gap finder

Analyse [module] and list its public functions with zero or weak test coverage. For each, tell me what the most valuable test would be - the one that would catch the most realistic future bug - and why. Do not write the tests yet.

Prioritisation before generation: you approve the plan before spending quota on tests that do not matter.

PROMPT 11 | The test suite writer

Write tests for [function/module] following the existing test conventions in this repo (check [test folder] first). Cover: happy path, each error branch, and edge cases [empty input, boundary values, concurrent calls - as relevant]. Run the tests, fix failures, show me the diff.

'Check the existing tests first' is the difference between tests that fit your suite and tests that fight it.

PROMPT 12 | The regression bodyguard

Here is the bug we just fixed: [description]. Write tests proving it cannot return, in the style of this repo. Then tell me which nearby code has the same class of bug and is still untested.

Bugs travel in families. This prompt finds the siblings.

Refactoring and reviewing (prompts 13-16)

PROMPT 13 | The safe refactor

Refactor [file/function] with NO behaviour change: [extract the validation into a helper / split this function / name this mess]. Keep the public interface identical. Run the full test suite before and after and show me both results plus the diff.

'No behaviour change' plus test-before-and-after is the safest refactor contract available to you.

PROMPT 14 | The pre-PR self review

Review the current diff as a skeptical senior engineer: correctness bugs, security issues (injection, auth, secrets), performance traps (N+1 queries, loops that allocate), missing tests, and violations of this project's conventions in CLAUDE.md. Rank findings by severity. Only report real issues - no style nitpicks already handled by the linter.

Run it before every PR. The severity ranking tells you what to fix now versus eventually.

PROMPT 15 | The fresh-eyes simplification

Show me the 3 places in [module] with the most unnecessary complexity - duplication, dead branches, over-abstraction. For each, show the simpler version. Do not change anything yet.

Complexity audit on demand; you decide what to actually simplify.

PROMPT 16 | The API contract checker

This is my intended change: [description]. List every other file, function or API endpoint that consumes what I am about to change, and what will break for each.

The blast-radius check that saves you from the 6pm surprise when CI goes red three services away.

Building (prompts 17-20)

PROMPT 17 | The spec-to-plan

Feature: [one-paragraph description]. Constraints: [performance / compatibility / style]. Read the relevant modules first, then present a step-by-step implementation plan with the files you will touch in order. Do not write code until I approve the plan.

This is chapter 9's Move 2 in prompt form - and 'do not write code until I approve' is the whole point.

PROMPT 18 | The thin slice

Implement step [n] of the approved plan only: [paste step]. Keep the diff minimal, run tests and lint, and report what changed in plain language.

Turns a scary 40-file plan into reviewable 30-minute steps.

PROMPT 19 | The CLI/script builder

Write a [language] script that: [task]. Requirements: arguments with --help, no hardcoded credentials (read from environment), error messages that say what to do next, and a usage example. Then run it once on sample input and show me the output.

'No hardcoded credentials' from the first draft, not as a cleanup later.

PROMPT 20 | The migration rehearsal

Plan a migration from [old library/DB/schema] to [new]. Stage it so each step is independently deployable and reversible. Identify the riskiest step and what a rollback looks like at that point. Do not start implementing.

Reversible steps and a named riskiest step - the two things panicky migrations skip.

Docs, commits and learning (prompts 21-25)**PROMPT 21 | The README that writes itself**

Write a README for this repo: what it is, quick start (commands from CLAUDE.md), configuration, and a short architecture section based on the actual directory structure. Match the repo's existing tone if it has any docs.

Because no one writes the README, and this version is grounded in the real commands.

PROMPT 22 | The commit and PR writer

From the current staged diff, write a conventional-commit message (max 72-char subject) and a PR description: what changed, why, how to verify, and any follow-ups deliberately not included. Base it only on the diff - do not invent context.

Boring, accurate, free. 'Only on the diff' keeps the description honest.

PROMPT 23 | The codebase Q&A; prep

I need to present [feature/module] to [audience: new team members / my lead / a client]. Generate the 10 questions they are most likely to ask, with answers grounded in this codebase.

Turns prep from a blank page into an editing task.

PROMPT 24 | The concept tutor

Explain [concept in this codebase or language] to me in three passes: (1) a 2-sentence intuition, (2) how it is actually used in THIS repo with file references, (3) the two most common misunderstandings about it. Then quiz me with 3 questions.

Pass 2 is the one a generic tutorial cannot give you - your own repo as the textbook.

PROMPT 25 | The code reviewer's debrief

Here is a PR review I just received: [paste comments]. For each comment, tell me if it reflects a real issue in the code, and show the specific fix. If a comment is wrong or a matter of taste, say so and explain why.

A second opinion on the first opinion - useful precisely because it disagrees with you when you are wrong.

CHAPTER 14

Review like your job depends on it

Agents write code in seconds. You are now the quality gate - here is how to be a good one.

The failure mode of agentic coding is not that the code is bad. It is that the code looks confident, the tests look thorough, and the diff is large - so tired-you hits approve. Every rule in this chapter exists to make approving safe.

The three-minute diff review

- 1 **Intent.** Does the diff do what the spec said - and only that? Drive-by changes go back out.
- 2 **Correctness.** Trace the main path by hand for 60 seconds. Agents fix the symptom you named and miss the adjacent bug surprisingly often.
- 3 **Security.** User input validated? Auth checks present on new endpoints? No secrets, keys or tokens added anywhere?
- 4 **Tests.** Do the new tests actually assert behaviour - or do they restate the implementation? Delete tests that cannot fail.
- 5 **Edges.** Empty input, boundary values, failure paths: which one did the agent not think of?
- 6 **Dependencies.** Any new package or import? Each one is a supply-chain decision - verify it or reject it.

Secrets and credentials

Zero tolerance, learned the hard way across the industry: no API key, token, password or connection string goes into a prompt, a file, or a commit - ever. Keys live in environment variables or a secrets manager; the agent reads them from there at runtime. If a key has ever been pasted into a chat, treat it as compromised and rotate it. And prompt 19's rule - no hardcoded credentials - goes into your context file so the agent holds the line even when you forget it.

Client code, NDAs and your employer's rules

Before any client or employer code touches any tool, know three things: whether the vendor trains on your data by default, how to turn that off (Cursor's privacy mode guarantees no training on your code; paid tiers generally opt out of training), and what your contract or company policy says. Many Indian enterprises route AI usage through approved enterprise configurations for exactly this reason - if yours does, use it; the enterprise version is usually more capable anyway. When in doubt, ask. 'The agent saw it' has never been a defence anyone accepted.

Spotting hallucinated code fast

- **Unknown imports and functions are the number one tell.** An unfamiliar symbol is either a hallucination or a part of the codebase you have not read - grep before you assume either.

- **Tests that pass suspiciously easily.** A test written to match the implementation always passes; a test written from the spec can fail. Rewrite tests that merely restate the code.
- **Confident prose around uncertain facts.** The comment that explains 'why' something works is the most hallucination-prone text in any diff - verify claimed behaviour against the actual code path.
- **The fix is procedural, not talent:** run the build, run the tests, and add 'verify all imports exist and do not guess APIs - say so when unsure' to your context file. Agents that are given permission to express uncertainty will use it.

The judgement you cannot delegate

Agents do not carry consequences; you do. Architecture decisions, security trade-offs, data-handling choices, anything with 'load-bearing' energy - those stay with you. A useful frame: the agent is the most brilliant junior engineer you have ever managed. You would never let a junior, however brilliant, single-handedly redesign your auth system at 2am and merge it unreviewed. Same rule.

How to read a 500-line diff in five minutes

- 1 **Read the spec or PR description first.** You are checking the diff against an intent, not auditing every line cold.
- 2 **Skim the file list before any content.** Files the spec never mentioned get full attention; expected files get normal attention.
- 3 **Read tests before implementation.** If the tests assert the right behaviour, the implementation review is a confirmation pass.
- 4 **Hunt the pattern, not the line:** search the diff for credential-shaped strings, new imports, deleted error handling, and changes to auth or payment paths - the four places where mistakes are expensive.
- 5 **Leave the style to the linter.** If your review comment could have been written by a machine, it should have been.

The flip side

Do not let review anxiety make you slow either. If you read every line of every diff with equal suspicion, agents make you a slower reader, not a faster shipper. The checklist above is deliberately short - three focused minutes beats thirty panicked ones, and the self-verification loop from chapter 9 means most breakage never reaches you.

CHAPTER 15

The money chapter

Free tiers first, quotas managed, and the real cost of a stack - in rupees.

AI tool pricing in 2026 is allowance-based: a headline price that includes a pool of usage, with extra consumption billed beyond it. That makes the headline a floor, not a ceiling. Here is how to keep the floor where it starts.

The five rules of AI tool spending

- 1 **Exhaust the free tiers first.** Gemini CLI's 1,000 requests/day, Copilot Free's 2,000 completions, Copilot Student for verified students, Cursor Hobby. For learning and side projects this is genuinely enough.
- 2 **Prefer rupee-billed plans where they exist.** Cursor Start at Rs. 649 tax-inclusive with UPI avoids both forex markup and the card-decline lottery that Indian developers know too well.

- 3 **Batch work into sessions.** Claude's rolling five-hour windows and Cursor's monthly pools both reward finishing a task in one sitting - re-reading a codebase three times a day triples your token spend for zero extra output.
- 4 **Watch the meter.** All three vendors expose dashboards (Cursor: cursor.com/dashboard; GitHub: your usage page; Claude: your account settings). A weekly glance beats a monthly surprise.
- 5 **Escalate when the tool saves you money, not when it is fashionable.** Upgrade when you hit limits on real work, not for status.

What a stack really costs

Stack	Monthly	Good for
Gemini CLI + Copilot Free	Rs. 0	Students, learning, side projects
Copilot Pro	~Rs. 830	Daily autocomplete, chat, model choice
Cursor Start (INR, UPI)	Rs. 649	Agent-first editing, rupee billing
Claude Pro (incl. Claude Code)	~Rs. 1,660	Terminal-first, repo-wide work
Cursor Start + Claude Pro	~Rs. 2,300	The two-tool power stack
Claude Max 5x	~Rs. 8,300	Heavy daily agent users

USD figures assume about Rs. 83 per dollar plus a 3% card forex charge; your bank's rate will differ slightly. Note the tax difference too: Start's Rs. 649 is tax-inclusive, the USD plans are not. The decision math for a working professional: senior Indian developer rates on the open market run Rs. 1,500-3,000 an hour. If the paid stack saves you one productive hour a day - and used with the workflow in chapter 9 it easily does - the payback period is measured in days.

The upgrade decision tree

- 1 **Are you hitting limits on real work, not exploration?** If no, you are done - free tiers plus one cheap plan is a legitimate permanent setup for many developers.
- 2 **Is the friction your tool or your workflow?** If outputs are wrong, re-read chapters 9 and 10 before paying for a different model - most 'tool quality' complaints are spec problems.
- 3 **Which single upgrade removes the most friction?** One move up (Free to Pro, Start to Pro, Pro to Max 5x) usually beats adding a second vendor.
- 4 **Only then consider the two-tool stack.** It is real money every month; it should solve a named, recurring pain - not boredom.

The quota triage, when you hit a wall

It is 9pm, your Claude five-hour window is exhausted, and the feature is due tomorrow. Move one: switch the task to Gemini CLI (1,000 free requests tomorrow, and its window resets daily). Move two: switch to a cheaper task class - autocomplete (Copilot) instead of agent mode for the mechanical part. Move three: batch it into tomorrow's first session and sleep - context files mean the agent restarts smart, not from zero.

CHAPTER 16

When the agent goes wrong

Seven failure modes you will meet in your first month - and the fix for each.

Every failure below has a fix that is already somewhere in this book. This chapter is the index you reach for at 11pm when something misbehaves.

Symptom	What is actually happening	The fix
The code calls an API that does not exist	Hallucination: the model invented a plausible function	Run the build; add 'verify all imports exist, do not guess APIs' to your context file
It edited the wrong file / broke something unrelated	Your spec had no scope boundary	Add a SCOPE section listing files in and out; use prompt 16 before risky changes
Quality drops in a long session	Context rot: the conversation outgrew what the model tracks well	Finish the task, start fresh; the context file carries what matters forward
It 'finishes' but tests were skipped or weakened	Agents optimise for the instruction 'make tests pass'	Require new tests to fail first (prompt 7); delete tests that cannot fail
It loops on the same error	Fixing symptoms of a root cause it has not found	Stop it; run prompt 6 or 7 (root-cause mode); restart from the new understanding
Everything is suddenly slow or capped	You hit a quota window (chapter 15)	Quota triage: switch tool, switch task class, or batch into the next window
The diff looks right but feels off	Plausible-but-wrong logic - the classic	The six-point diff review (chapter 14); trace the main path by hand before approving

Notice the pattern: six of the seven fixes are better instructions, not a better tool. That is the reframe that separates people who bounce off agentic coding from people who keep it. When the output is bad, the first thing to fix is usually the input - the spec, the scope, the verification loop - and only then the vendor.

CHAPTER 17

Your first 30 days

A week-by-week plan that ends with agentic coding as your default, not your experiment.

Week 1 - Set up and explore (cost: Rs. 0)

- Install Gemini CLI and log in with your Google account. Run the codebase map (prompt 1) on any project you own.
- Install the Copilot extension in your editor and use the free tier's completions for the week.
- Read one official doc page end-to-end: your tool's quickstart. The documentation for Claude Code, Cursor, Copilot and Gemini CLI is good - use it.
- Checkpoint: you have run an agent in your terminal and an assistant in your editor, on your own machine.

Week 2 - The workflow (cost: Rs. 0-650)

- Pick your primary tool by the profiles in chapter 8 (most readers: Cursor Start, or Copilot Pro if it is already at your office).
- Write your first CLAUDE.md / rules file using the chapter 10 template - on a real project, not a toy.

- Do one real ticket with the full loop: explore, spec, plan, small diff, self-verify, review, commit (prompts 1, 17, 18, 22).
- Checkpoint: one merged PR where the agent wrote most of the code and you understood every line.

Week 3 - Depth (cost: same as week 2)

- Add automation: hooks or settings that run your formatter/linter after every agent edit.
- Run the pre-PR self review (prompt 14) on every diff this week - including ones you wrote by hand.
- Try the test suite prompts (10-12) on your most under-tested module.
- Checkpoint: your team's review comments on your PRs have visibly changed in kind, not just volume.

Week 4 - Integration and judgement (cost: same)

- Do one task you have been avoiding for months - the docs rewrite, the dependency upgrade, the flaky-test hunt. These are exactly what agents eat.
- Time yourself honestly: what used to take a day now takes what? That ratio, not hype, is your answer to whether to keep paying.
- Review your usage dashboard once, and right-size your plan with chapter 15.
- Checkpoint: agentic coding is how you work by default, and you can explain every line you shipped this month.

The same plan, day by day

Day	Focus	Done when
1-2	Install Gemini CLI + Copilot Free; run prompt 1 on any repo you own	You have seen an agent read real code
3-4	Read your primary tool's official quickstart end to end	You can name its three main features
5-7	Pick your primary tool by chapter 8; write your first context file	CLAUDE.md/rules exists in a real project
8-10	One real ticket with the full loop (prompts 17, 18, 22)	A merged PR you understand line by line
11-14	Add hooks/format-on-edit; run prompt 14 on every diff this week	Self-review is now a habit, not an event
15-18	Test-suite prompts (10-12) on your most under-tested module	A module's coverage visibly improved
19-21	Try a second tool free-tier for a day; decide keep-or-drop	You know why your primary is your primary
22-25	The task you have avoided for months (docs, upgrade, flaky tests)	It is off your list
26-28	Usage dashboard review; right-size your plan (chapter 15)	You pay for exactly what you use
29-30	The honest test below	Three yeses - or a workflow fix

The one-month honest test

On day 30, ask: (1) Am I shipping more of what matters? (2) Do I understand my codebase better or worse than a month ago? (3) Would I be comfortable defending every diff in a review? Three yeses - keep going and consider the two-tool stack. Any no - return to chapters 9 and 14; the workflow, not the tool, is usually what needs fixing.

CHAPTER 18

The team playbook

Rolling out AI coding tools across a team - pilots, guardrails, and what to measure.

Individual adoption is chapter 17's problem. This chapter is for the tech lead or manager deciding how the whole team adopts - because with 90% of professional developers already using these tools weekly, the question at work is no longer whether, but how.

Run a real pilot, not a committee

Pick one project and 5-8 volunteers for four to six weeks of real work - not a toy sandbox. Choose the vendor by the same logic as chapter 8's team profile: predictable per-seat pricing and admin controls matter more at scale than peak capability. Before the pilot starts, freeze the guardrails: privacy/training settings configured, the secrets policy from chapter 14 stated, and the review checklist added to the PR template so it is enforced by process, not memory.

Measure two numbers, ignore the rest

- **Review-rework rate:** how much of what the agent wrote survives review untouched. Falling rework means the specs and context files are working; flat rework means people are prompt-throwing.
- **Time-to-merge:** for comparable ticket classes before versus after. It should fall; if it does not, find where it went (usually review becoming the new bottleneck - which means the team needs chapter 14's reading skills, not a newer tool).
- What to ignore: lines of code, number of prompts, 'AI-written percentage'. All three rise with bad habits as easily as good ones.

The three objections you will hear

- 1 **'Code quality will drop.'** Point at the rework-rate metric you are tracking, and at the review checklist now in the PR template. Quality is a process property; the pilot measures it.
- 2 **'Juniors will stop learning.'** Pair every junior with the review discipline from day one: they read and defend every diff the agent produces. Juniors who review heavily graduate faster - they see more code in a month than their predecessors saw in a year.
- 3 **'Our code cannot leave the premises.'** Then the pilot runs on the enterprise configurations built for exactly this - and the vendor's privacy commitments go in writing. This objection is why per-seat enterprise plans with admin controls exist.

The quiet first move

Before any of it: check what the company already pays for. Many Indian IT firms and GCCs hold Copilot seats or enterprise AI licences that teams simply do not use - JetBrains found Copilot adopted by 40% of developers at companies over 5,000 employees. Activating what is bought is cheaper than piloting what is not.

CHAPTER 19

What this means for your career

Augmentation beats automation - and the market is already paying for the difference.

The fear that 'AI will replace developers' has aged badly. What the 2026 data shows instead is a widening gap between developers who direct AI well and developers who do not use it at all - and employers are paying for the difference. GitHub's Octoverse found nearly 80% of new developers adopt Copilot in their first week; the incoming generation treats agents as the default the way your generation treated Stack Overflow. The differentiator is not access - everyone has access - it is workflow.

Anthropic's economic research classifies AI usage as augmentation (you learn, iterate, decide) versus automation (you delegate end-to-end). The same research that found India leading the world in software-related AI usage also found Indian usage still concentrated - a small professional segment, largely in four states, using these tools intensively. That concentration is your opportunity: the practices in this book are not yet table stakes in most Indian engineering teams, and the developers who bring the workflow - spec-writing, context files, review discipline, CI integration - become the people who raise their team's output, not just their own.

What to actually get good at

- **Specification writing.** The skill that replaces fast typing. Clear goals, scoped files, stated constraints, defined success. If you can write a good bug report, you are already halfway there.
- **Reading code faster than you write it.** Review becomes your bottleneck skill; invest in it deliberately.
- **Testing.** Agents generate code at unbounded speed; tests are what make that speed safe. Engineers who think in testable contracts thrive.
- **Architecture and domain depth.** The agent can implement any approach; only you can choose the right one. Domain knowledge is what your prompt is worth.
- **Security judgement.** Chapter 11's checklist is the minimum. As code volume explodes, the people who can vouch for code become the senior people.

And the meta-skill over all of them: comfort with tool churn. Copilot led this market for two years; Claude Code went from launch to the most-used AI coding tool at work in about one; Cursor's India user base tripled in a year; Google ships a competitive CLI as open source. The tools you use in 2027 will differ from the ones in this book. The workflow - explore, specify, plan, verify, review - will not. Learn the workflow and the tools become interchangeable parts.

Quick-reference checklists

Print this page, or keep it open in a second monitor tab forever.

Before you let an agent touch code

- Context file exists and is current (CLAUDE.md / GEMINI.md / rules).
- Spec written: goal, files in scope, constraints, how to verify.
- Agent presented a plan and you interrogated it.

- No secrets, keys or client-confidential data in reach of the session.
- Task is one small diff, not 'rewrite the module'.

Before you approve any diff

- It does what the spec said - and nothing else.
- You traced the main path by hand.
- Input validated, auth checked, no secrets introduced.
- New tests can actually fail; they assert behaviour, not implementation.
- Edge cases considered: empty, boundary, failure.
- No new dependency without your explicit approval.

The weekly 15-minute maintenance

- Skim your usage meter: are you over- or under-using what you pay for?
- Add anything a review caught this week to the context file - it compounds.
- Glance at your tool's changelog or release notes - features arrive monthly in this market.
- Delete or archive prompts and one-off scripts that did not survive contact with real work.

Before you pay for any plan

- You have hit the limits of the free tiers on real work, not toy work.
- You checked what your employer already provides.
- You know what your plan's allowance is, and where the meter lives.
- Rupee-billed options compared (Cursor Start) against USD plans plus forex.
- Privacy/training settings configured for the code you will actually work on.

The command cheat sheet

What	Where / how
Install Claude Code (macOS/Linux/WSL)	<code>curl -fsSL https://claude.ai/install.sh bash</code>
Start Claude Code in a project	<code>cd your-project && claude</code>
Claude Code in the browser	claude.ai/code
Install Gemini CLI	<code>npm install -g @google/gemini-cli</code> (Node 20+)
Gemini CLI one-off question	<code>gemini -p "Explain this codebase"</code> (add <code>--output-format json</code> for scripts)
Project context files	CLAUDE.md (Claude Code), GEMINI.md (Gemini CLI), rules (Cursor)
Cursor India plan	cursor.com/pricing - Start, Rs. 649/mo tax-inclusive, UPI accepted
Copilot free tier	2,000 completions/month; students get full features after verification
Usage meters	cursor.com/dashboard (Cursor); settings/usage on claude.ai and github.com
Automated PR review	Claude Code or Gemini CLI in GitHub Actions (chapter 12)

Glossary

The twelve terms this book uses, in one place.

- **Agent (coding agent):** an AI tool that plans and executes multi-file tasks - editing code, running commands, iterating - rather than only suggesting text.
- **Autocomplete / completion:** real-time inline suggestions as you type; unlimited on paid Copilot plans, and the layer every tool still offers.
- **Context window:** how much text the model can hold in one session; '1M tokens' means the agent can read a large codebase in one go.
- **Context file:** a project-root markdown file (CLAUDE.md, GEMINI.md, rules) read at session start - your persistent instructions to the agent.
- **Hallucination:** confident output that is wrong - invented APIs, misremembered syntax. Fixed by verification loops, not by trusting harder.
- **MCP (Model Context Protocol):** the open standard for connecting agents to external systems - Jira, Slack, databases, internal tools.
- **Privacy mode:** a vendor guarantee (Cursor's is contractual on its pricing page) that your code is not used for model training.
- **AI credits:** GitHub Copilot's usage meter for chat and agent workloads; completions stay unlimited on paid plans.
- **Quota window:** a rolling usage limit - Claude's five-hour session windows, Cursor's monthly pool, Gemini CLI's 1,000 requests/day.
- **Spec:** the written task description you give an agent: goal, scope, constraints, verification. The highest-leverage minute in agentic coding.
- **Diff:** the set of changes the agent proposes; the thing you review, always.
- **Hook:** a shell command that runs automatically before/after agent actions - format on edit, lint before commit.

Sources

Every fact in this book traces to one of these official pages, checked September 2026.

- GitHub Octoverse 2025 - 'A new developer joins GitHub every second as AI leads TypeScript to #1', github.blog, 28 October 2025 (India developer counts, growth, projections, Copilot first-week adoption).
- JetBrains AI Pulse - 'AI Coding Agents: Adoption Trends', blog.jetbrains.com/research, 18 August 2026 (90% weekly agent use, 68% daily, Claude Code 39%, Copilot 21%, Cursor 12%, Antigravity 15% in India).
- Anthropic - 'India Country Brief: Anthropic Economic Index', anthropic.com, 16 February 2026 (5.8% share of Claude.ai use, 45.2% software tasks, task-level breakdown, task compression, state distribution).
- Anthropic - 'Anthropic raises \$30B Series G funding', anthropic.com, 12 February 2026 (Claude Code \$2.5B+ run-rate revenue, weekly active users doubled, 4% of public GitHub commits estimate).
- Anthropic - Claude Code documentation, docs.anthropic.com (installation, surfaces, CLAUDE.md, hooks, skills, MCP, parallel and background agents, routines).
- Anthropic - Plans and Pricing, anthropic.com (Free/Pro/Max pricing and inclusions, September 2026).

- Cursor - Pricing, cursor.com/pricing and cursor.com/docs/models-and-pricing; 'Introducing Cursor Start', cursor.com/blog, 28 July 2026 (Start Rs. 649 tax-inclusive, UPI, included and excluded features, Pro/Pro+/Ultra/Tiers, privacy mode, India user base).
- GitHub - 'Plans for GitHub Copilot', docs.github.com (Free, Student, Pro, Pro+, Max, Business, Enterprise pricing and AI credits; model list; September 2026).
- Google - Gemini CLI documentation and README, google-gemini.github.io/gemini-cli (free tier 60 req/min and 1,000 req/day, Gemini 2.5 Pro, 1M-token context, GEMINI.md, MCP, Apache 2.0).

Prices, limits and features in this book were verified against these sources in September 2026 and will change over time. Before paying for anything, re-check the vendor's official pricing page - it is linked above. Nothing in this book is affiliated or sponsored; no vendor paid for inclusion.

Where to keep learning - the primary sources

The best ongoing resource is not a blog or this book - it is each vendor's own documentation, which is genuinely good and updated continuously: the Claude Code docs and quickstart at docs.anthropic.com and code.claude.com/docs; Cursor's documentation, help centre and forum (including the Start announcement post) at cursor.com; GitHub Copilot's get-started guides at docs.github.com; and the Gemini CLI documentation and repository at google-gemini.github.io/gemini-cli. Bookmark the four, and re-read your tool's changelog monthly - this market moves faster than any book can follow.